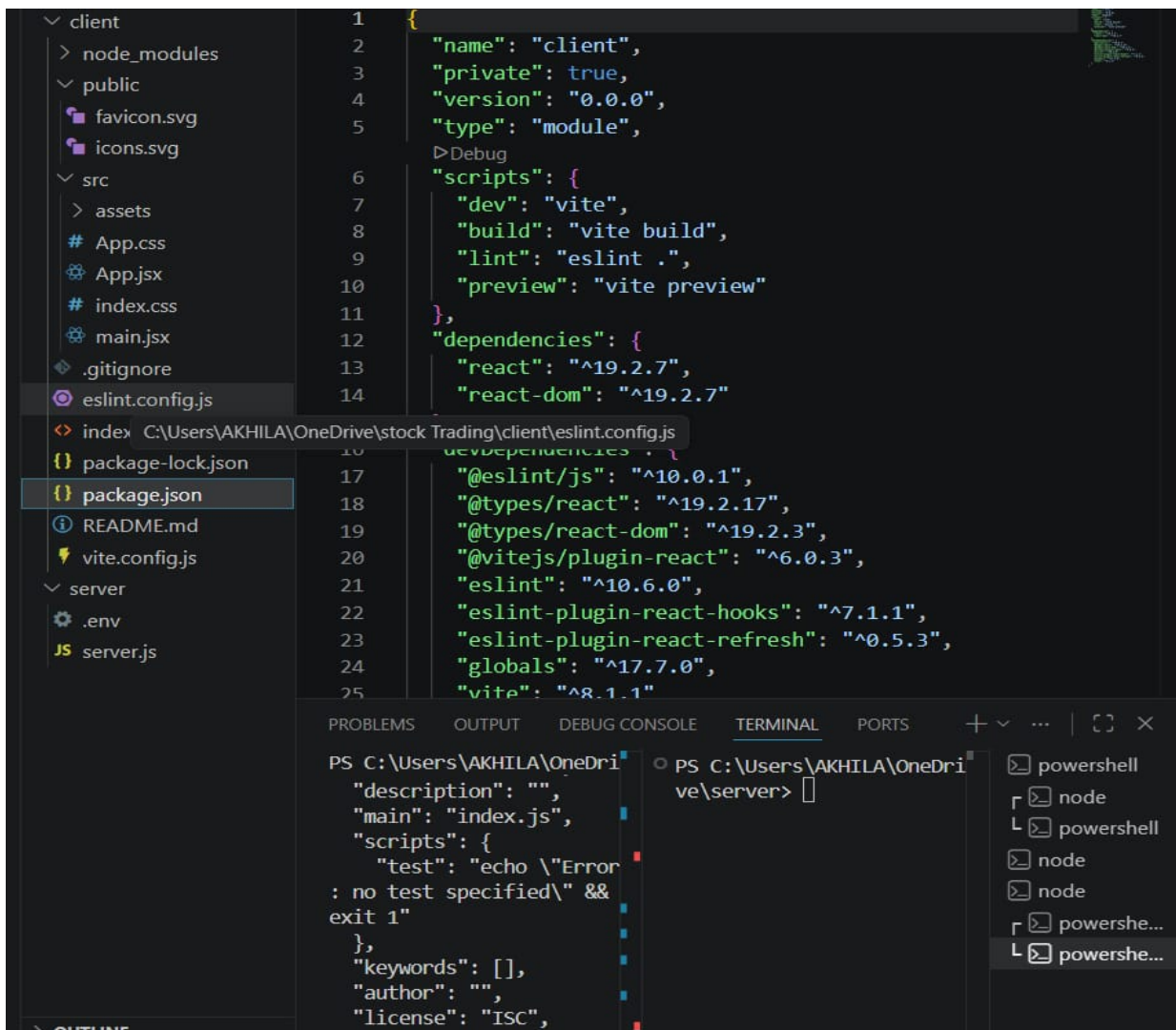


BACKEND DEVOLPMENT

Page 1: Server Setup

The backend of the Stock Trading App was developed using Node.js, Express.js, MongoDB Atlas, and Mongoose. The server folder was initialized separately using npm. Essential packages such as express, cors, dotenv, mongoose, and nodemon were installed. A package.json file was created to manage dependencies and scripts. The project structure includes config, controllers, middleware, models, routes, uploads, utils, server.js, package.json, and .env.

The server was started using nodemon with the command 'npm run dev'. Environment variables were stored securely in the .env file including PORT and MONGO_URI. This setup provides a scalable and maintainable backend foundation.



Page 2: MongoDB Connection

MongoDB Atlas was used as the cloud database. A free cluster was created and a database user was configured. The connection string was stored in the .env file. A db.js file was created inside the config folder. The connectDB() function establishes the database connection using mongoose.connect().

If the connection is successful, the application prints 'MongoDB Connected' with the host name. If the connection fails, an error message is displayed and the application exits. This ensures reliable database connectivity before processing requests.

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver

Version

Node.js

6.7 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```



[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String

☐ View full code sample

```
mongodb+srv://Akhila:<db_password>@cluster0.sk4lmo.mongodb.net/?  
appName=Cluster0
```



Replace **<db_password>** with the password for the **Akhila** database user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Node.js Driver](#)

[Node.js Starter Sample App](#)

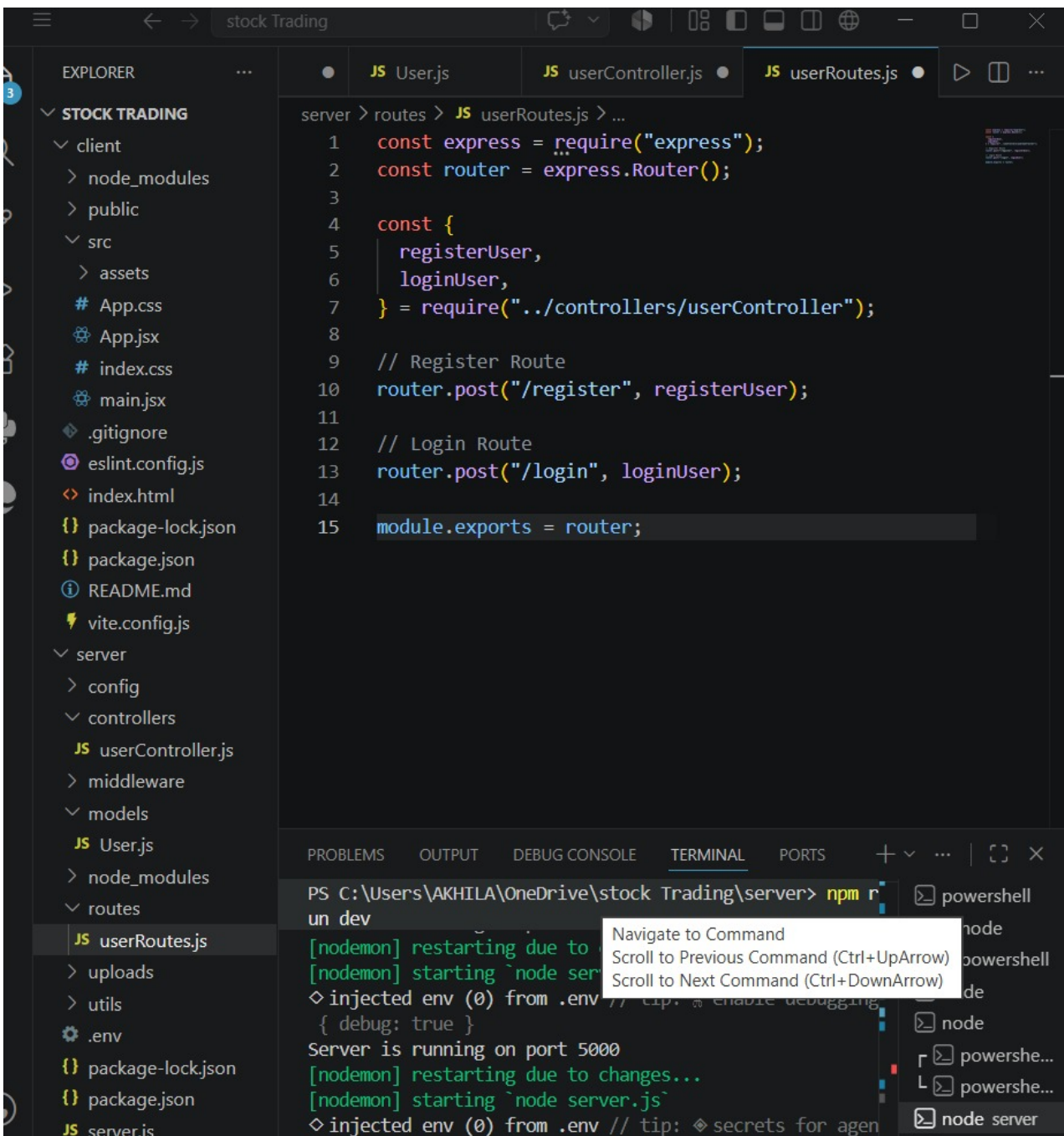
[Access your Database Users](#)

[Troubleshoot Connections](#)

Page 3: Express Server and Routing

The Express server loads environment variables, connects to MongoDB, enables CORS, and parses JSON requests. User routes are registered using `app.use('/api/users', userRoutes)`. The root route displays a backend status message.

The `userRoutes.js` file defines API endpoints such as `POST /register` and `POST /login`. These routes forward requests to the appropriate controller functions, providing a clean MVC architecture.



The screenshot shows a Visual Studio Code editor with a project named "stock Trading". The Explorer panel on the left shows the project structure, including folders for client, src, server, and routes. The file `userRoutes.js` is selected in the routes folder. The main editor displays the code for `userRoutes.js`, which uses Express.js to define routes for user registration and login. The code is as follows:

```
1 const express = require("express");
2 const router = express.Router();
3
4 const {
5   registerUser,
6   loginUser,
7 } = require("../controllers/userController");
8
9 // Register Route
10 router.post("/register", registerUser);
11
12 // Login Route
13 router.post("/login", loginUser);
14
15 module.exports = router;
```

Below the code editor, the TERMINAL panel shows the command prompt output. The user has run `npm run dev` in the directory `C:\Users\AKHILA\OneDrive\stock Trading\server`. The output shows that the server is running on port 5000. A tooltip is visible over the terminal, providing navigation instructions: "Navigate to Command", "Scroll to Previous Command (Ctrl+UpArrow)", and "Scroll to Next Command (Ctrl+DownArrow)".

Page 4: Controllers and API Testing

The `UserController.js` file contains the business logic. During registration, the controller receives the user's name, email, and password, checks whether the email already exists in MongoDB, and creates a new user if it does not exist. Appropriate success and error responses are returned in JSON format.

Thunder Client was used to test backend APIs. POST requests were sent to `http://localhost:5000/api/users/register` with JSON data. Successful responses confirmed that routing, controllers, Express, and MongoDB integration were working correctly. This backend implementation follows the MVC pattern and serves as the foundation for authentication and future stock trading features.

